Internship Report On

“NTPWEdit Clone - Windows Password Management Tool”

In the partial fulfillment of the requirement for Third Year Computer Engineering

Submitted by Kakad Saurabh Madhukar Roll No: 32 Rohan Sachin Pagar Roll No: 32

Under the Guidance of Prof. P. P. Kakade

DEPARTMENT OF COMPUTER ENGINEERING

NGSPM’S BRAHMA VALLEY COLLEGE OF ENGINEERING AND RESEARCH INSTITUTE NASHIK-422213

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE 2025-2026

---

# ACKNOWLEDGEMENT

It is a great pleasure for us to express our deep sense of gratitude to all those who have helped us directly and indirectly in the successful completion of this internship project.

We would like to express our special thanks of gratitude to our guide, **Prof. P. P. Kakade**, for their valuable guidance and constant encouragement during the development of this project. Their insights and technical expertise were instrumental in overcoming the various challenges we faced.

We also express our sincere thanks to the **H.O.D. of Computer Engineering Department** and the **Principal of Brahma Valley College of Engineering and Research Institute, Nashik**, for providing us with the necessary facilities and a conducive environment for our project work.

Finally, we would like to thank our parents and friends for their continuous support and motivation.

**Kakad Saurabh Madhukar Rohan Sachin Pagar**

---

# ABSTRACT

The Security Accounts Manager (SAM) hive is a critical component of the Windows NT operating system family, storing sensitive user account information, including password hashes and group memberships. Management of these accounts typically requires the operating system to be active; however, in scenarios involving system lockouts or forensic investigations, offline access is required.

This internship project, titled **"NTPWEdit Clone"**, involves the development of a lightweight, standalone C++ application designed to parse and manipulate Windows Registry hives directly at the binary level. By bypassing the Windows Registry APIs, the tool can operate on SAM files located on non-booted drives, such as those accessed via Windows Recovery Environment (WinRE) or a bootable Linux/Live PE environment.

The project demonstrates a modular approach to system-level programming, featuring a custom registry hive parsing engine, a SAM-specific data manager, and a native Win32 user interface. The report details the technical implementation of registry record structures (nk, vk, sk, etc.), the logic behind user account enumeration, and the challenges of building a dependency-free system tool using the MinGW toolkit. The result is a functional utility that automates SAM detection and provides a clear interface for account visualization and planned password management.

---

# INDEX


| Sr. No. | Content | Page No. |
|---|---|---|
| **1** | **INTRODUCTION** | **1** |
| 1.1 | Project Overview | 1 |
| 1.2 | Problem Statement | 2 |
| 1.3 | Objectives of the Project | 3 |
| 1.4 | Scope of the Project | 4 |
| 1.5 | Methodology | 5 |
| **2** | **LITERATURE SURVEY / BACKGROUND** | **7** |
| 2.1 | Evolution of the Windows Registry | 7 |
| 2.2 | The Security Accounts Manager (SAM) | 8 |
| 2.3 | Existing Solutions and their Limitations | 10 |
| **3** | **REGISTRY HIVE FORMAT DEEP DIVE** | **12** |
| 3.1 | Structural Overview | 12 |
| 3.2 | The REGF Header | 14 |
| 3.3 | HBIN Blocks and Cells | 16 |
| 3.4 | Key Nodes (nk) and Value Keys (vk) | 18 |
| 3.5 | Subkey Lists (lf, lh, ri, li) | 20 |
| **4** | **SYSTEM FLOW AND ARCHITECTURE** | **23** |
| 4.1 | Overall System Flow | 23 |
| 4.2 | Component-Based Architecture | 25 |
| 4.3 | Functional Flow Diagrams | 27 |
| **5** | **TECHNICAL IMPLEMENTATION - REGISTRY ENGINE** | **30** |
| 5.1 | Registry Engine (ntreg.cpp) | 30 |
| 5.2 | Cell Level Access and Pointers | 32 |
| 5.3 | Enumerate Subkeys Logic | 34 |
| 5.4 | Value Key Retrieval | 36 |
| **6** | **TECHNICAL IMPLEMENTATION - SAM MANAGER** | **38** |
| 6.1 | SAM Management (sam.cpp) | 38 |

| | | |
|---|---|---|
| 6.2 | Path Resolution in SAM | 40 |
| 6.3 | User Account Enumeration | 42 |
| 6.4 | Forensic Analysis of V and F records | 44 |
| **7** | **WIN32 UI ARCHITECTURE** | **47** |
| 7.1 | UI Design Philosophy | 47 |
| 7.2 | Main Dialog Implementation | 49 |
| 7.3 | ListView Data Binding | 51 |
| 7.4 | Automatic Path Detection | 53 |
| **8** | **BUILD SYSTEM AND AUTOMATION** | **55** |
| 8.1 | MinGW Toolkit Integration | 55 |
| 8.2 | Resource Compilation with windres | 57 |
| 8.3 | Batch Build Logic | 59 |
| **9** | **RESULTS AND PERFORMANCE** | **62** |
| 9.1 | Testing Methodology | 62 |
| 9.2 | Functional Verification | 64 |
| 9.3 | Performance Benchmarks | 66 |
| **10** | **DISCUSSION AND SECURITY** | **68** |
| 10.1 | Security Implications | 68 |
| 10.2 | Mitigation Strategies | 69 |
| 10.3 | Technical Challenges | 70 |
| **11** | **CONCLUSION** | **72** |
| **12** | **FUTURE SCOPE** | **74** |
| **13** | **APPENDICES** | **76** |
| 13.1 | Appendix A: Source Code (ntreg.h) | 76 |
| 13.2 | Appendix B: Source Code (sam.h) | 79 |
| 13.3 | Appendix C: Resource Script | 82 |
| **14** | **REFERENCES** | **85** |


---

# CHAPTER 1: INTRODUCTION

## 1.1 Project Overview

The Windows Registry is a hierarchical database that stores configuration settings and options on Microsoft Windows operating systems. It contains settings for the hardware, operating system software, most non-operating-system software, users, preferences of the PCs, etc. The registry provides a window into the operation of the runtime system.

One of the most critical parts of the registry is the **Security Accounts Manager (SAM)**. The SAM hive stores local user accounts, passwords (in hashed format), and group memberships. Under normal operating conditions, the SAM hive is locked by the Windows kernel (System process) and cannot be read or modified by standard user-mode applications or even Administrators using the standard `Regedit` tool.

The **"NTPWEdit Clone"** project is an endeavor to create a standalone, high-performance C++ application that can read and potentially modify these locked files by accessing them "offline." This means the tool can be used when the operating system is not running, or when the drive is mounted on another system. The clone replicates the core functionality of the original NTPWEdit utility while being built from the ground up with a custom registry parsing engine.

## 1.2 Problem Statement

In the modern computing environment, access control is paramount. However, several scenarios arise where the standard access control mechanisms become a hindrance:

1. **Forgotten Credentials**: Users or administrators frequently forget local account passwords, leading to locked systems and data loss.
2. **Administrative Lockouts**: Security policies or malware can lead to the disablement of critical administrative accounts.
3. **Forensic Investigation**: Security professionals often need to audit account activity and status on dead disks or forensic images without altering the system state via a live boot.
4. **Registry Corruption**: Standard OS tools often fail to handle slightly corrupted hives, requiring a more resilient, low-level parser.

## 1.3 Objectives of the Project

The primary objectives of the NTPWEdit Clone system are:

1. **Develop a Robust Registry Parser**: Create a C++ engine capable of navigating binary hives.
2. **Bypass Kernel Locks**: Implement file-based access that works on non-active hives.
3. **Automate Identification**: Implement logic to automatically find the local system's SAM file.
4. **Enumerate Accounts**: Correctly extract user names and their corresponding Relative Identifiers (RIDs).
5. **Clean Native UI**: Provide a responsive Win32-based interface for user interaction.

---

# CHAPTER 2: BACKGROUND

## 2.1 The Windows Registry History

The registry was first seen in Windows 3.1 but was greatly expanded in Windows 95 and Windows NT. It replaces the old `.ini` files used in Windows 3.x. The move to a binary database allowed for better performance, security, and the ability to store complex data types.

## 2.2 Security Accounts Manager (SAM)

The SAM is a database file that stores users' passwords. It is used to authenticate local and remote users. It uses cryptographic measures to prevent unauthorized access. The file is located at `%SystemRoot%\System32\config\SAM`.

---

# CHAPTER 3: REGISTRY HIVE FORMAT DEEP DIVE

## 3.1 Structural Overview

A registry hive is an on-disk file that contains a discrete body of registry keys, subkeys, and values. The format is a complex system of "Cells" and "Bins."

### 3.1.1 Hierarchical Structure

The registry is structured like a file system:

- **Keys** are like directories.
- **Values** are like files.
- **Data** is the content of the "files".

## 3.2 The REGF Header

The `regf` header is at the very beginning of the file (first 4KB).

| Field | Offset | Size | Description |
|---|---|---|---|
| id | 0 | 4 | Magic number "regf" |
| seq1 | 4 | 4 | Sequence number |
| ver_major | 20 | 4 | Major version |
| size | 40 | 4 | Total hive size |
| checksum | 508 | 4 | Header XOR checksum |

## 3.3 HBIN Blocks and Cells

Blocks are 4096-aligned units. Each block contains multiple cells.

- **Cells** are the actual data units.
- **Signature**: Cells are preceded by their size (negative = alloc, positive = free).

---

# CHAPTER 5: TECHNICAL IMPLEMENTATION

## 5.1 Registry Engine (ntreg.cpp)

The implementation of the `Hive` class is where the majority of the low-level complexity resides.

### 5.1.1 Cell Level Access

Every offset in the registry is relative to the start of the hbin section (4096 bytes).

```
void* Hive::getOffset(DWORD offset) {
    if (offset == 0xFFFFFFFF) return nullptr;
    return (char*)buffer + 4096 + offset + 4;
}
```

---

# CHAPTER 6: SAM MANAGER

## 6.1 User Account Enumeration

The SAM hive stores users in `SAM\Domains\Account\Users\Names`. Each key name here is a username.

## 6.2 Forensic Analysis

The "V" and "F" records contain the core meta-data:

- **F-Record**: Timestamps and account control bits.
- **V-Record**: Username and encrypted password hashes.

---

# CHAPTER 13: APPENDICES

## 13.1 Appendix A: Source Code (ntreg.h)

```
#ifndef NTREG_H
#define NTREG_H

#include <windows.h>
#include <vector>
#include <string>

#pragma pack(push, 1)
struct regf_header {
    char id[4]; // "regf"
    DWORD seq1;
    DWORD seq2;
    FILETIME mtime;
    DWORD ver_major;
    DWORD ver_minor;
    DWORD type;
    DWORD unknown1;
    DWORD root_key_offset;
    DWORD size;
    DWORD unknown2;
    char name[64];
    char unknown3[396];
    DWORD checksum;
};
```

```
struct nk_record {
    char id[2]; // "nk"
    WORD flags;
    FILETIME mtime;
    DWORD unknown1;
    DWORD parent_offset;
    DWORD subkeys_count;
    DWORD unknown2;
    DWORD subkeys_list_offset;
    DWORD unknown3;
    DWORD values_count;
    DWORD values_list_offset;
    DWORD sk_offset;
    DWORD class_offset;
    WORD name_len;
    WORD class_len;
    char name[1];
};
#pragma pack(pop)

class Hive {
public:
    Hive();
    bool open(const std::wstring& path);
    void* getOffset(DWORD offset);
    DWORD findSubkey(std::string name, DWORD parent_nk = 0);
    std::vector<std::pair<std::string, DWORD>> enumerateSubkeys(DWORD nk_offset);
    void* getValueData(DWORD nk_offset, std::string valueName, DWORD* type = nullptr, DWORD*
len = nullptr);

private:
    void* buffer;
    DWORD bufferSize;
};

#endif
```

## 13.2 Appendix B: Source Code (sam.h)

```
#ifndef SAM_H
#define SAM_H

#include "ntreg.h"

struct UserEntry {
    std::string username;
    DWORD rid;
    std::string status;
    DWORD nk_offset;
};
```

```
class SAMManager {
public:
    SAMManager(Hive* hive);
    std::vector<UserEntry> loadUsers();
    bool unlockAccount(DWORD rid);
    bool changePassword(DWORD rid, std::string newPassword);

private:
    Hive* hive;
    DWORD findUserNK(DWORD rid);
};

#endif
```

---

# GLOSSARY OF TERMS

- **ACB (Account Control Bits)**: A bitmask in the SAM hive that controls account properties like "Disabled" or "Locked Out."
- **Cell**: The smallest unit of data storage in a Registry hive (header + data).
- **Hive**: A file containing a discrete body of registry data.
- **NK (Named Key)**: A record type representing a registry key.
- **NT Hash**: An MD4-based hash used by Windows for user authentication.
- **RID (Relative Identifier)**: The last part of a SID that uniquely identifies an account within a domain (e.g., 500 for Administrator).
- **SAM (Security Accounts Manager)**: The portion of the registry that manages user and group security.
- **VK (Value Key)**: A record type representing a registry value and its data.

---

# REFERENCES


1. Microsoft Corporation, "Windows Registry Reference".
2. Russinovich, M., "Windows Internals".
3. Nordahl-Hagen, B., "Offline NT Password & Registry Editor".
4. Forensics Wiki, "Registry Hive Format".